Minimum Viable Specification for a Worldwide Agentic Web

Draft for web implementers and agent system builders

MVP Profile 0.2 | 6 November 2026 | Cognizant AI Lab

Purpose This document defines the minimum interoperable behavior needed to publish and run portable multi-agent systems on the existing web. A conforming agent site publishes a data-only network definition. A standard web browser runtime runs that network with the site's default inference configuration or, when offered, a visitor-selected router and credential. With the visitor's approval, one agent site can load a second site's published agents and run them in the caller's browser.

The profile uses ordinary HTTPS hosting, browser security boundaries, CORS, JavaScript modules, JSON Schema, and MCP. It does not require a new transport, a central registry, or a server-side agent runtime. The goal is a small target that independent teams can implement on the existing web and extend through compatible services, tools, and clients.

Table of Contents

1. SCOPE AND CONFORMANCE 2

1.1 Intended Use and Boundaries 2
1.2 Terms 3

2. TENET ONE: COMMON MULTI AGENT REPRESENTATION 3

2.1 Minimum Network Content 3
2.2 Portable Data Profile 4

3. TENET TWO: BROWSER EXECUTION AND SITE COMPOSITION 4

3.1 Minimum Runtime Behavior 4
3.2 Protocol for Loading a Peer Agent Site 5
3.3 Minimum Public Files 6
3.4 Browser Origin Requirements 7

4. TENET THREE: PLUGGABLE INFERENCE 7

5. CROSS CUTTING SECURITY AND PRIVACY REQUIREMENTS 7

6. MVP INTEROPERABILITY CHECKS 8

7. EXTENSIONS AND COMMUNITY OPPORTUNITIES 8

8. REFERENCE IMPLEMENTATIONS AND SOURCE MATERIAL 9

1. Scope and Conformance

This profile defines three core tenets and the publication, privacy, and safety rules that make them interoperable. An implementation may use different internal libraries or agent frameworks and it conforms when it produces and consumes the same public artifacts and observable behaviors.

This draft records the minimum profile implemented by a reference runtime.

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are to be interpreted as described in RFC 2119 and RFC 8174.

For this profile, an agent-site manifest (agent-manifest.json) is a public, machine-readable index of the agent networks published by a site. It identifies each network, its front agent, and the location of its resolved network definition, and may include site-level inference, capability, policy, and deployment metadata. The resolved network definition is the data-only HOCON or JSON artifact containing the portable agent graph, with build-time includes and substitutions expanded. Peer-link metadata maps named references to other published agent-network URLs, while peer-policy metadata states which peer origins may be contacted. CORS-enabled hosting allows browsers on other origins to fetch these public artifacts.

Capability	Minimum requirement	Interoperability result
Portable network	Neuro SAN HOCON data model with a resolved, data-only web artifact	The same multi-agent graph can move between publishers and runtimes
Browser execution	JavaScript orchestration, validation, tool calls, and linked-site loading	A normal browser is the agent container
Pluggable inference	Site-provided default router and model with optional visitor Bring-Your-Own-Key (BYOK)	Inference funding and deployment can change without changing the agent graph
Web publication	Manifest, resolved network definition, peer-link and peer-policy metadata, and CORS-enabled hosting	A caller can locate and validate a peer without a central registry
Trust boundary	Consent, browser and CORS boundaries, private-context filtering, and execution limits	A linked site cannot silently acquire the user's router credential or unlimited execution

1.1 Intended Use and Boundaries

This version targets open, portable agent networks that a normal web page can publish and a visitor's browser can run. The published HOCON, including agent instructions, is public by design. This lets users and developers inspect behavior, compare implementations, test portability, and build compatible runtimes. Publisher-funded and user-funded model access both fit this profile. A publisher that needs confidential instructions, controlled data, or secure execution can keep that functionality server-side and expose a typed capability through authenticated MCP, a site action, or a hosted agent protocol. Authentication, billing, and service-level policies for those services are outside the minimum requirements.

- **Tab-scoped execution:** The browser runtime operates for the lifetime of the page. Scheduled, event-driven, background, or durable tasks require a hosted runtime or another service.
- **Direct linking:** A user or an agent in the caller network may initiate a request to use another published agent network. The caller site's browser runtime resolves a direct URL or a publisher- or user-supplied named link to a specific network at a peer agent site and, after required user approval, loads that network into the caller's browser.
- **Publisher operations:** Analytics, rate limits, monetization, liability allocation, and commercial terms are publisher concerns or future metadata, not interoperability requirements.

1.2 Terms

- **Agent network:** A named graph of agents, their instructions, invocation schemas, and available agents or tools.
- **Agent site:** A web origin or subpath that publishes one or more agent networks and the files needed to run them.
- **Caller site:** The site and browser runtime handling the user's current session.
- **Peer site:** A second agent site whose network is loaded by the caller.
- **Front agent:** The public entry point for an agent network. Other published agents support the network but are not its external entry point.
- **LLM router:** A site-provided or user-selected inference service that directs model requests to one or more models through an interface supported by the runtime. This profile does not mandate a particular inference wire protocol.
- **Site action:** A capability owned by a web site and explicitly registered with the browser runtime. Registration provides its name, purpose, input schema, effect classification, and handler. A local site action executes within, or invokes services belonging to, the caller's origin. A cross-site action is owned by another origin and requires a transport mechanism, origin checks, authorization, and any required user confirmation; cross-site action transport is outside the minimum profile.
- **Private context (sly_data):** Structured data carried alongside an agent invocation but outside its ordinary request and response text. It may contain credentials, session context, or other sensitive values. Access to private-context fields is controlled through explicit allow rules.

The reference implementation currently supports OpenAI-compatible endpoints through its LLM client. Other protocol adapters, local model servers, and browser-based inference engines may satisfy the same interface.

2. Tenet One: Common Multi Agent Representation

The canonical representation MUST follow the Neuro SAN Agent Network HOCON File Reference as a working schema for the agent network description. HOCON is the canonical authoring and exchange representation. Because JSON is a valid HOCON serialization, a site MAY publish an equivalent, fully resolved JSON document for browser loading. The published form MUST preserve the Neuro SAN data model and MUST NOT depend on local includes, environment substitutions, or publisher-only file paths that the Neuro SAN HOCON format affords on server systems.

2.1 Minimum Network Content

- **Identity and metadata:** A stable network name, a short description, and the name of its front agent.
- **Agent definitions:** Each agent has a unique name within the scope of a single agent network hocon description, a function description, instructions when it uses an LLM, and a list of agents or tools it may call.
- **Invocation schemas:** Callable agents and tools expose JSON Schema-compatible inputs. Structured outputs SHOULD also declare schemas.
- **Model settings:** Model names and generation settings are hints, and the caller's active router and model policy take precedence when a peer runs in the browser. A compatible extension may let a publisher declare tested or permitted model lists, model families, or capability ranges and let the runtime warn or refuse when the selected model falls outside that policy.
- **Private context rules:** Private data is separate from chat content and is deny-by-default across agent-site boundaries. A network must explicitly name fields that may pass downstream or upstream.
- **External capabilities:** An agent may reference another published agent-network URL or an MCP server. An MCP descriptor may restrict the tool names exposed to the agent. A runtime MAY also expose a catalog of registered site actions. An agent may propose a call to an exact catalog entry; the runtime validates the arguments, obtains any required user confirmation, and dispatches the call to the site-owned handler. Cross-origin access remains subject to browser CORS and origin policy. A declared external capability may therefore be unavailable at runtime unless its origin permits the caller or an authorized same-origin proxy or hosted bridge is used. The runtime MUST fail closed and report unavailable capabilities clearly; it MUST NOT bypass browser security controls.

2.2 Portable Data Profile

The minimum interoperable network definition MUST be declarative data. Its published instructions and schemas form a public interoperability surface and MUST NOT be treated as a confidentiality mechanism; reuse remains subject to the publisher's license and terms. A coded-tool bundle MAY be included when required by an agent network's functionality, but it is not part of the minimum portable data profile. A packaging step MAY resolve HOCON includes, reusable definitions, and framework-specific configuration into the published network and MAY package coded tools separately. Confidential or server-side capabilities MAY remain outside the published network and be exposed through MCP, authenticated web services, or another hosted agent protocol. Browser-local capabilities MAY be exposed through runtime-registered site actions. Public executable capabilities MAY use the optional coded-tool mechanism implemented by the reference runtime.

```
{
  "metadata": {
    "description": "Local services guide"
  },
  "tools": [
    {
      "name": "guide",
      "function": {
        "description": "Finds the right local service"
      },
      "instructions": "Understand the request and delegate it",
      "tools": [
        "lodging",
        "https://peer.example/transit"
      ]
    },
    {
      "name": "lodging",
      "function": {
        "description": "Answers lodging questions"
      },
      "instructions": "Use only grounded lodging information",
      "tools": [
        {
          "url": "https://tools.example/mcp",
          "tools": ["search"]
        }
      ]
    }
  ]
}
```

Other agent frameworks available in the browser space can map into this representation. Google ADK names, descriptions, instructions, sub-agents, tools, models, and input or output schemas map to core HOCON constructs. Framework-specific workflows, lifecycle hooks, state, or custom runtime behavior may require an adapter or may not be portable. A converter or Skill file may automate the mapping; conformance is determined by the emitted HOCON profile, not by the source framework. See the Google ADK agent configuration documentation and a reference implementation for a convertor.

3. Tenet Two: Browser Execution and Site Composition

A conforming runtime MUST be usable from a normal web page and MUST execute the published graph in JavaScript. The nsDeploy browser runtime is one reference implementation. It parses and validates a network, runs the front agent and internal agents, translates callable capabilities into model-visible function schemas, dispatches approved calls, and returns structured results to the parent agent.

3.1 Minimum Runtime Behavior

- **Parse and validate:** Reject malformed networks, duplicate or missing agent references, an invalid front agent, and unsupported executable definitions before any model call.
- **Run the agent loop:** Support model streaming, tool calls, agent-to-agent delegation, error propagation, and bounded conversation history.

- **Use grounded capabilities:** Support MCP as the baseline interface for tools and data. Ask for user approval before discovering or calling a third-party MCP service.
- **Handle local site actions:** Support same-site registered actions described by name, purpose, JSON Schema, effect, and handler. The runtime MUST validate arguments and show the user the destination and arguments before a write or irreversible operation. Cross-site action transport is an extension.
- **Report outcomes accurately:** Distinguish a returned HTTP or tool response from a verified business outcome. Timeouts and disconnects after a write begins MUST be reported as having an unknown outcome. This does not prohibit retries performed within the invoked service under its configured retry and idempotency policies; however, the browser runtime MUST NOT independently repeat a write whose outcome is unknown.
- **Bound execution:** Enforce finite limits on composition depth, agent invocations, remote calls, concurrency, and elapsed execution time. Limits SHOULD be configurable by network, subject to runtime-defined maximums. A runtime MAY detect or prevent cycles when appropriate to its execution model; repeated invocation of the same agent MUST NOT, by itself, be treated as a cycle.

3.2 Protocol for Loading a Peer Agent Site

This is the minimum agent-site-to-agent-site protocol. It composes agent definitions, not remote execution services. The peer's agents run in the caller's browser.

1. **Reference:** A peer network is identified by an HTTP or HTTPS URL whose final path component is its published network name. In version 0.1, the runtime treats the preceding path as the site base and retrieves `agent-manifest.json` from that base. For example, https://peer.example/agents/transit identifies the transit network listed in https://peer.example/agents/agent-manifest.json. The manifest is part of the site's published bundle and maps network names to their data artifacts and associated metadata. Named links and same-site networks MAY resolve to URLs using the same convention. HTTPS is required except during local development.
2. **Local execution:** The caller instantiates the peer network in the caller's runtime. The peer uses the caller's active LLM router, model policy, and execution limits. When a user key is active, it is sent only to that router and never to the peer site. A network MAY publish recommended or required model capabilities, or a list of supported models; interpreting and enforcing those declarations is an optional extension.
3. **Model compatibility policies:** Define declarations and negotiation mechanisms through which a network can identify required model capabilities, minimum operating characteristics, or supported models, allowing a runtime to select a compatible model or warn the user before execution.
4. **Consent:** An interactive runtime MUST provide a consent mechanism governing contact with third-party origins. By default, it MUST identify the external origin and purpose of the contact and obtain the user's approval before each contact. A runtime MAY support alternative trust policies and consent interfaces, but any relaxation of the default MUST result from an explicit user choice.
5. **Published network resolution:** An external-network URL identifies a site base and a requested network name. The runtime retrieves the site's static `agent-manifest.json` and selects the entry whose name exactly matches the requested network. If no exact match exists, it MUST fail rather than select another published network. The manifest is part of the site's published file bundle; no application server or dynamic discovery service is required.
6. **Definition fetch:** The runtime resolves the manifest's hocon value against the peer site base, fetches the referenced network definition, and validates it before execution. The value may be a relative path or an absolute HTTP or HTTPS URL. This version follows normal browser fetch redirects and relies on browser CORS enforcement; it does not require the final network-definition URL to remain on the manifest's origin.
7. **Result return: Result return:** The peer front agent's answer and declared structured output return through the caller's normal tool-result path. Private context MUST NOT return through that path except for fields explicitly permitted by the caller network.
8. **Interactive MCP authentication:** Standardize browser-mediated acquisition, storage, refresh, and endpoint-scoped use of MCP credentials, including OAuth or SSO redirects. Private context could supply authorized HTTP metadata to a specific MCP server without exposing credentials to agent prompts, peer sites, or unrelated services.
9. **Per hop policy:** Every additional peer or MCP hop repeats the origin-policy and consent checks. The runtime stops cycles and requests that exceed its budgets.

MVP boundary: A conforming runtime MUST be able to retrieve and instantiate the declarative network definition identified by a peer-network URL. Conformance does not require downloading or executing publisher-supplied code. Cross-origin artifacts MUST be published with appropriate CORS headers. Support for coded-tool

bundles is optional; a runtime that supports them MUST treat them as untrusted code and apply the isolation and integrity controls described above. Cross-origin invocation of site-registered actions is an extension and is not required for MVP conformance. Such actions remain on their owning origin and retain that origin's authentication and authorization controls.

3.3 Minimum Public Files

Resource	Required content	Hosting rule
agent-manifest.json	One or more networks with name, front_man, and hocon; top-site LLM router metadata; peer policy; optional brand, MCP, linking, deployment, action, theme, and coded-tool metadata	Public GET from the derived site base; CORS enabled for callers
Network definition	Resolved document conforming to the Neuro SAN agent-network schema, serialized as JSON for browser loading	HTTP(S) location declared by the manifest; public GET access. When retrieved from another origin, the publisher MUST permit browser access using standard cross-origin controls.
links.json (peer-network link map)	Publisher-owned default peer-network bindings that map symbolic external-network references in a local network definition to concrete peer-network URLs	Public GET; browser-local user overrides are stored separately by the caller origin
Optional coded-tool bundle	Self-contained JavaScript module and optional SRI digest declared by the manifest	Public GET; if executed, run in an isolated browser Web Worker with runtime-enforced CSP and capability restrictions

Example peer-network link map: Suppose the `local_services` network refers to an external network by the symbolic name `transit`. Its published links.json can resolve that name to a concrete peer-network URL:

```
{
  "version": 1,
  "networks": {
    "local_services": {
      "transit": "https://peer.example/agents/transit"
    }
  }
}
```

This file provides address resolution for named peer-network references.

Optional discovery metadata: A publisher MAY expose site identity and exported-network metadata under `.well-known/nsdeploy/`. These files are not required for this version's conformance or peer-network execution.

The reference implementation commonly publishes a resolved network definition as `<network>.network.json`. This filename is a convention, not an interoperability requirement; runtimes use the location declared in the site manifest.

The publisher owns the default peer-network link bindings in the public links.json. A binding supplied at runtime for an unresolved named peer-network reference is caller-owned and MUST NOT modify the publisher's file. The reference runtime stores such bindings in origin-scoped browser storage and may persist them through a dynamic site endpoint; a static deployment does not rewrite the published file. The specification does not prescribe how a runtime obtains a missing binding. Only public agent artifacts require cross-origin access. Action modules, authentication endpoints, and business operations remain on their owning origin and retain their normal browser security policies. Production deployments MUST use HTTPS; loopback HTTP is acceptable for local development.

The optional reference implementation's `.well-known` files provide discovery metadata only. They are not an alternate execution path. A site may separately publish an A2A Agent Card at `/.well-known/agent-card.json` for A2A interoperability; this version's nsDeploy loader does not consume that card.

3.4 Browser Origin Requirements

Conformance does not require a cross-origin deployment. Networks and supporting artifacts MAY be co-bundled or hosted under the caller's origin, in which case their retrieval does not require CORS.

When a browser retrieves a public agent artifact from another origin, the publishing origin MUST permit that retrieval through standard browser cross-origin controls. Public artifact requests MUST NOT include the caller's private credentials.

Direct browser access to a cross-origin MCP server requires that server, or an authorized proxy, to permit the required origins, methods, and headers. Site actions and their authentication flows remain on their owning origin and MAY instead use origin-validated browser messaging. A business request made by such an action to its own origin does not require CORS.

Optional coded-tool bundles follow the same retrieval rules as other public artifacts and remain subject to the isolation and integrity requirements defined above.

4. Tenet Three: Pluggable Inference

The agent graph is independent of the service that performs inference. A site MAY publish a default router and model, including a fixed endpoint operated or funded by the publisher. It MAY also offer Bring Your Own Key (BYOK) so the visitor supplies a router, model, and credential. The reference implementation supports both modes through the nsDeploy browser engine, its BYOK profiles, and its runtime-supported router.

- **Default inference:** A site MAY publish a default router and model that work without a visitor-supplied key. The site MAY absorb inference cost, authenticate the browser through its own session, or route to infrastructure it controls.
- **Optional BYOK:** A runtime can conform without offering BYOK when the published default inference path runs the network. When BYOK is offered, the user MUST be able to select or enter the permitted router base URL, model, and credential, subject to any supported site model policy.
- **Credential handling:** When BYOK is active, the key remains in the caller's browser and is attached only to model requests sent to the selected router. A conforming runtime MUST NOT place it in URLs, manifests, agent network specification, prompts, action arguments, traces, analytics, or peer-site requests.
- **Router-only disclosure:** The top site selects the active default or BYOK router. A loaded peer inherits that router; its manifest cannot replace the router or receive a user key. Any router metadata present in a peer manifest is informational during composition and MUST NOT override the caller's active inference path.
- **Caller-site BYOK proxy option:** The site hosting the interactive runtime MAY expose a same-origin forwarding endpoint when a visitor-selected LLM router does not permit direct browser access through CORS. In BYOK mode, this endpoint forwards the visitor's authorization header and MUST NOT substitute a publisher-owned provider key. Peer sites are not contacted for inference and MUST NOT receive the visitor's credential. A separate default-inference endpoint MAY use publisher-controlled credentials.
- **Static BYOK option:** A static bundle MAY call a runtime-supported (e.g., OpenAI-compatible) router directly when that router permits browser CORS. When the selected router requires a key, a missing key must stop the model call with a clear setup error.
- **Reference storage:** When BYOK is used, the reference runtime stores profiles, including credentials, in origin-scoped localStorage. On loopback hosts only, it may mirror the profile to a host cookie so settings follow local ports. The user MUST be able to delete stored profiles.
- **Threat boundary:** BYOK can remove routine credential custody and inference cost from a publisher's server, but it does not isolate the credential from other JavaScript executing on the caller origin. Publishers SHOULD minimize third-party scripts and deploy a restrictive Content Security Policy.

5. Cross Cutting Security and Privacy Requirements

- **Untrusted instructions:** Peer-network definitions MUST be treated as untrusted input to the caller's inference model. Although the declarative format prevents those definitions from executing directly as code,

publisher-supplied agent instructions can still attempt to manipulate the caller's model—for example, by overriding caller policies, eliciting information available to the model, or inducing unsafe tool or action calls. Schema validation and coded-tool sandboxing do not establish trust in the peer network or its publisher.

- **Optional peer code:** A runtime MAY support coded-tool bundles. A bundle MAY be co-published with a local network or retrieved from a location declared by a peer manifest; the runtime MUST NOT execute undeclared peer code. Cross-origin retrieval is subject to standard browser access controls. A supporting runtime MUST execute the bundle only within the isolated Web Worker environment described above. When the manifest supplies a supported SRI digest, the runtime MUST verify it before execution and MUST reject the bundle if verification fails. Supplying a digest is optional in this version.
- **Explicit destinations:** Before an unauthorized third-party contact, the consent mechanism MUST identify the destination origin and the nature of the intended request. A redirect to a different origin MUST be treated as a new destination and evaluated under the same consent policy.
- **Consent before contact:** **Potentially** interactive approval policy precedes third-party manifest fetches, MCP discovery, and MCP calls. A remembered approval is scoped to the destination origin for the current session. Consent records a user's decision; it is not evidence that peer instructions or model-generated descriptions are safe.
- **Private context by exception:** Private-context fields MUST NOT pass from one network invocation to another by default. Only explicitly named fields MAY be forwarded under an applicable allow rule. The delegated request text is ordinary input to the destination network and is not filtered as private context.
- **Destination-scoped private-context policies:** Bind private-context permissions to a specific destination network identity rather than an entire site origin, allowing independently governed networks hosted on the same domain to have separate trust policies.
- **MCP disclosure:** A third-party MCP service receives the model-generated tool arguments approved for its call. MCP authentication, OAuth, and service-specific credentials are separate from the LLM BYOK key and are not transferred automatically.
- **Fail closed:** If a network, tool, or action invocation cannot be safely validated, resolved, or authorized, or exceeds an applicable execution limit, the runtime MUST NOT begin or continue that invocation and MUST NOT represent it as successful. The failure MUST be reported to the invoking network. This does not require terminating the overall request; the network MAY recover, select an alternative, or return a partial result according to its fault-tolerance policy. A remote timeout MUST be reported as a failure or, where side effects may already have begun, as an unknown outcome.
- **User-owned writes:** The runtime, not the model, owns confirmation. The executed action, origin, arguments, request identifier, and receipt are bound to the reviewed proposal.
- **Observable execution:** Implementations SHOULD expose the selected router, contacted origins, agent and tool calls, consent decisions, errors, and action receipts without exposing secrets.

6. MVP Interoperability Checks

An implementation is interoperable when it can demonstrate all of the following without publisher-specific integration code.

- Publish a two-agent HOCON network and a manifest from an ordinary HTTPS web host.
- Load and run that network in a browser with the site's published default router and model; if BYOK is offered, also demonstrate the user-selected path.
- Call an internal agent and at least one grounded capability through MCP or a registered local site action.
- After user approval, load an exact named network from a second origin and run it locally in the caller's browser.
- When a user key is active, show that the second site never receives it and cannot change the caller's router.
- Show that private context is absent at the peer unless a named field is explicitly allowed.
- Stop an invalid network, a composition cycle, an over-budget request, and a denied origin with visible errors.

7. Extensions and Community Opportunities

These current prototypes and natural extensions are outside the minimum profile for portable agent data, browser execution, and pluggable inference.

- Transactions negotiation and arbitration Policy-bound mandates, sealed bids, deterministic negotiation, settlement controls, and auditable receipts through a trusted arbiter.

- Cross-site browser actions Site-owned action catalogs, origin-checked iframe or popup bridges, caller permissions, confirmation, idempotency, and unknown-outcome handling.
- Private stewards — Operator-hosted Neuro SAN networks that keep credentials and private policy on the steward's server while loading and running public agent networks locally, with origin-level controls and an MCP compatibility fallback.
- CRUSE generative interfaces Agent-proposed forms, choices, reports, tables, charts, and action controls that are validated and rendered by the browser runtime.
- ANDe assisted HOCON authoring Natural-language creation and modification of HOCON networks through the Neuro SAN Agent Network Designer, usable through clients such as MAUI.
- MAUI client A web client for hosted Neuro SAN systems and agent-network authoring experiences.
- Framework converters Skills and tools that translate Google ADK and other framework graphs into the portable HOCON profile and verify semantic equivalence.
- Authenticated MCP and hosted bridges OAuth, user or service credentials, hosted MCP gateways, and policy controls for private or commercial data sources.
- A2A discovery and hosted execution Standard Agent Cards, registries, authenticated remote agents, durable tasks, background work, scheduling, and operation beyond a browser tab's lifetime.
- Local and browser inference Local model servers and WebGPU or WebAssembly engines such as WebLLM could implement the inference interface without a provider key when they meet the site's model and tool policy.
- Versioning integrity and publisher terms Profile fields, hashes, caller pinning, signatures, licensing, attribution, and re-consent when published artifacts change.

Note that while examples of agentic sites linked from this spec tend to take the shape of chatbots, they need not necessarily be invoked in this manner. For example, one can conceive of agents auto redesigning a web page as it is being used, through analysis of implied context and user behavior.

Additional work includes model setting extensions, WebMCP integration, client-server agent extensions, registries and search, identity and reputation, model compatibility, capability negotiation, accessibility, conformance tests, consent policies, cost controls, caching, analytics, rate limits, agent harness extensions, looping and event-based agent implementations, sly_data inter-agent data transport, and profile governance.

8. Reference Implementations and Source Material

- nsDeploy. Reference browser runtime for publishing, linked sites, pluggable inference, BYOK, MCP, and actions.
- Neuro SAN Agent Network HOCON File Reference. Normative common agent-network data model.
- Neuro SAN Studio. Agent Network Designer, examples, and hosted development tooling.
- A2A Protocol. Optional Agent Card discovery and hosted agent interoperability.
- Worldwide agentic web prototype demonstration. Current research prototype demonstration.